

AIAA 2205: Introduction to AI

Enyan Dai

The Hong Kong University of Science and Technology (Guangzhou)

Multi-Layer Perception

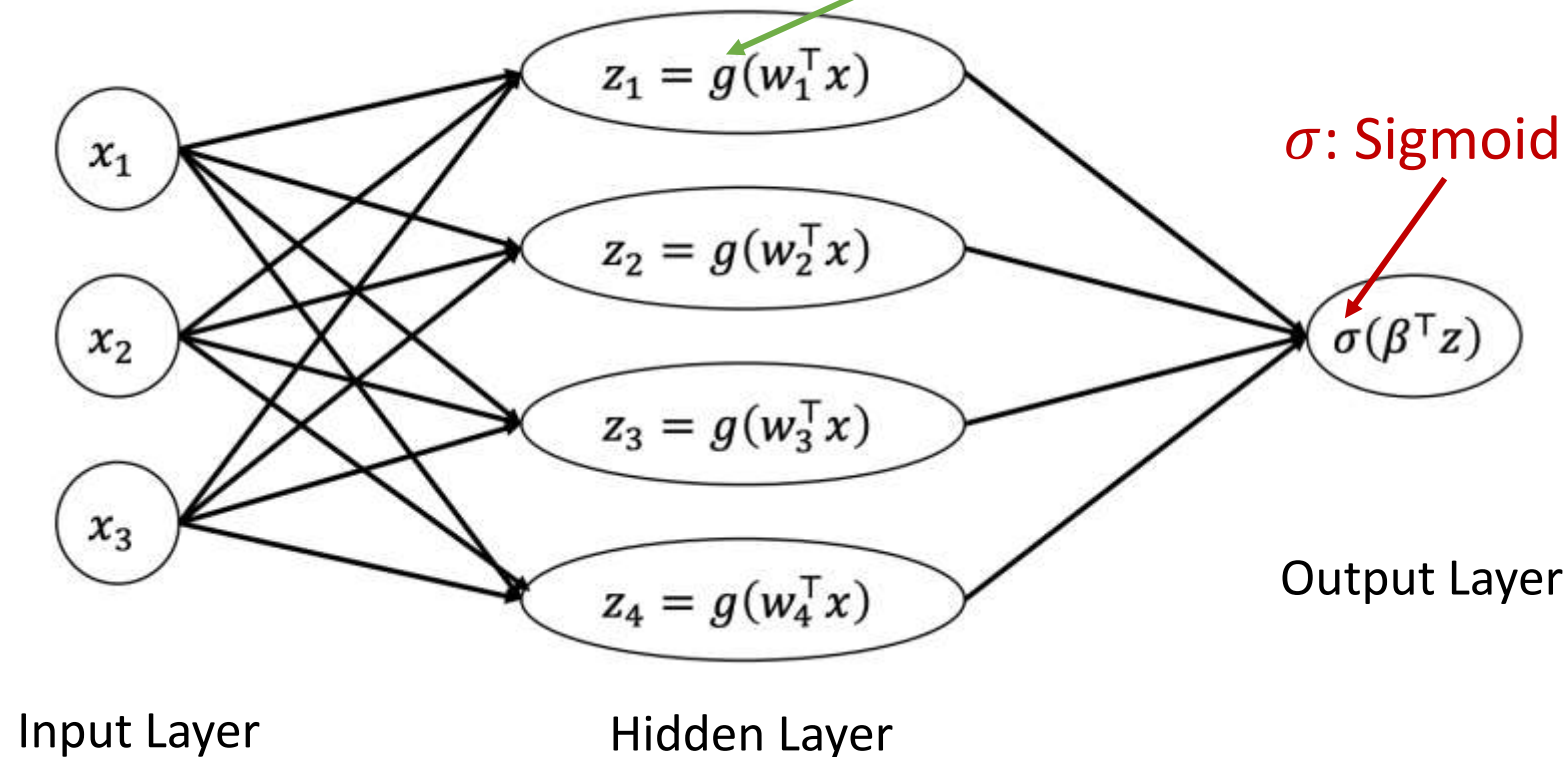
MLP is composed of multiple layers of neural networks

For binary classification: $f(\mathbf{x}) = \sigma(\beta^T g(\mathbf{W}\mathbf{x}))$

g : Non-linear activation function

σ : Sigmoid activation

MLP can stack many hidden layers

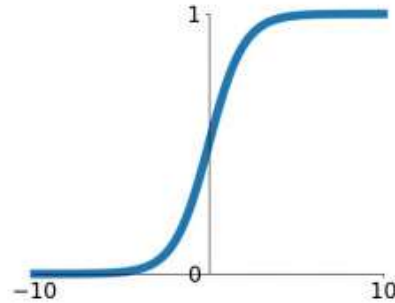


Activation functions

ReLU is a good default choice for most problems!

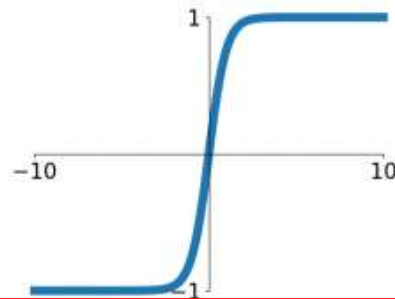
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



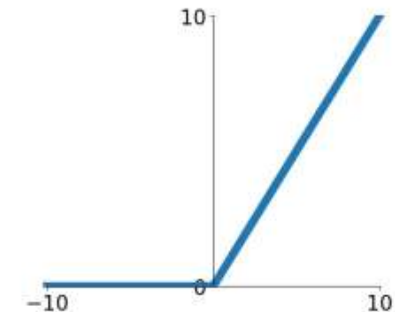
tanh

$$\tanh(x)$$



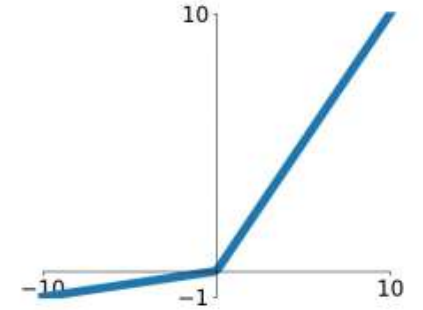
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

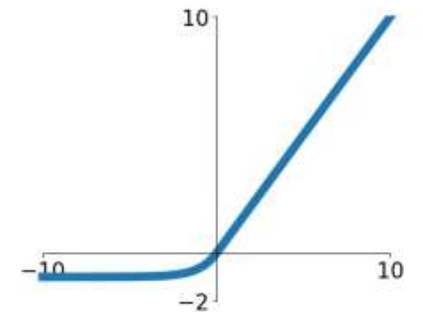


Maxout

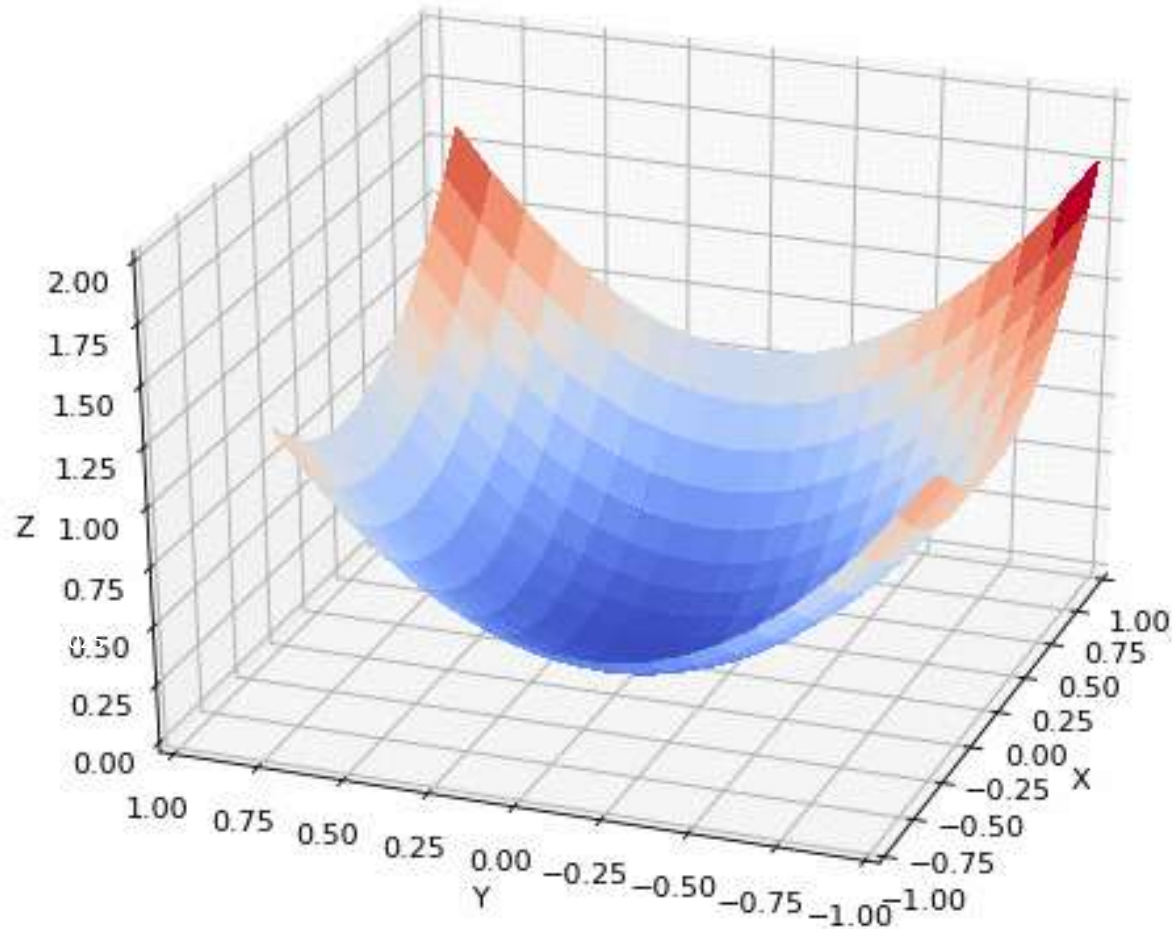
$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Gradient descent



Idea: follow the direction that leads to steepest loss reduction.

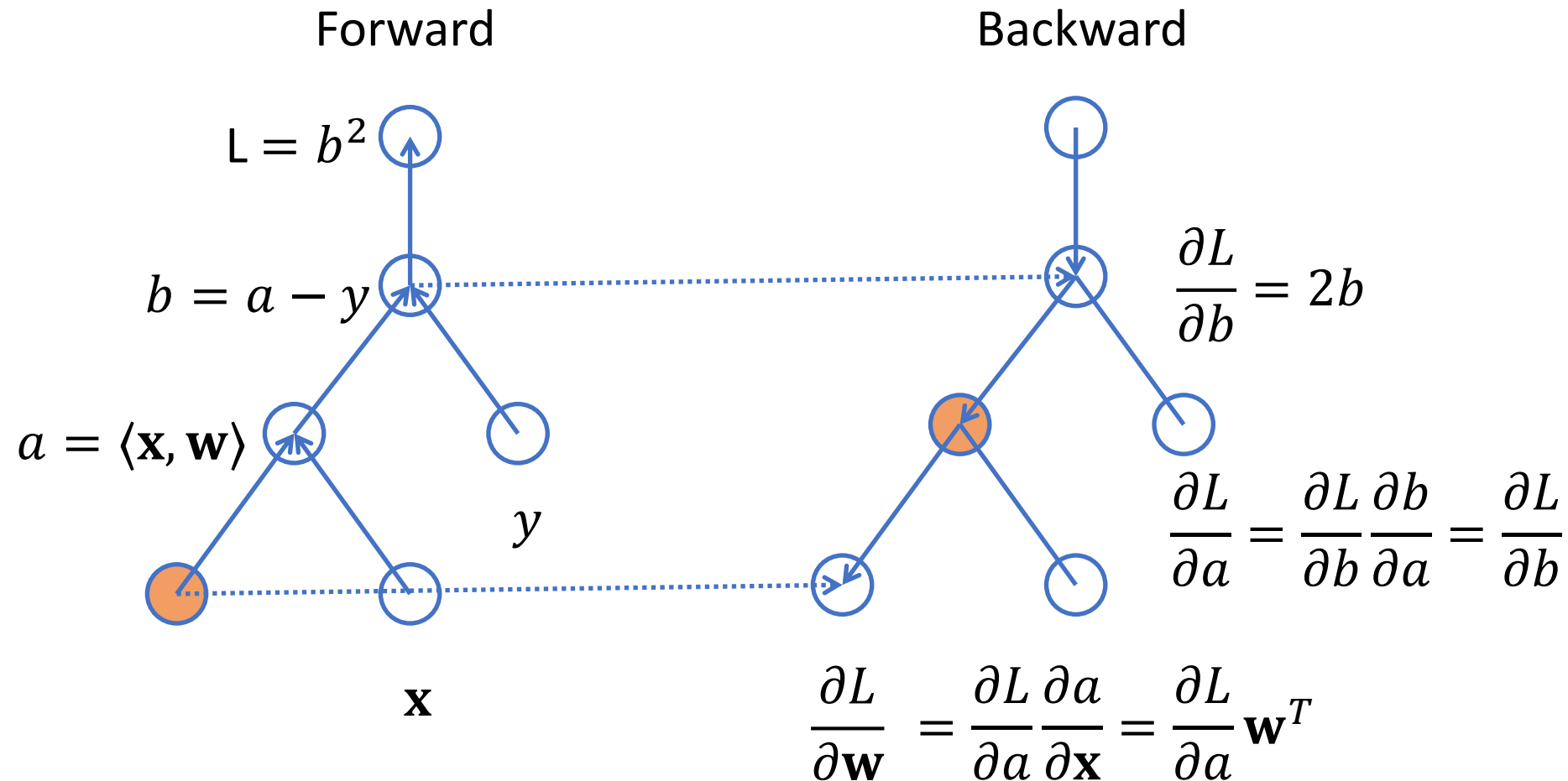
$$\mathbf{W} \leftarrow \mathbf{W} - \lambda \frac{\partial L}{\partial \mathbf{W}}$$

Step size

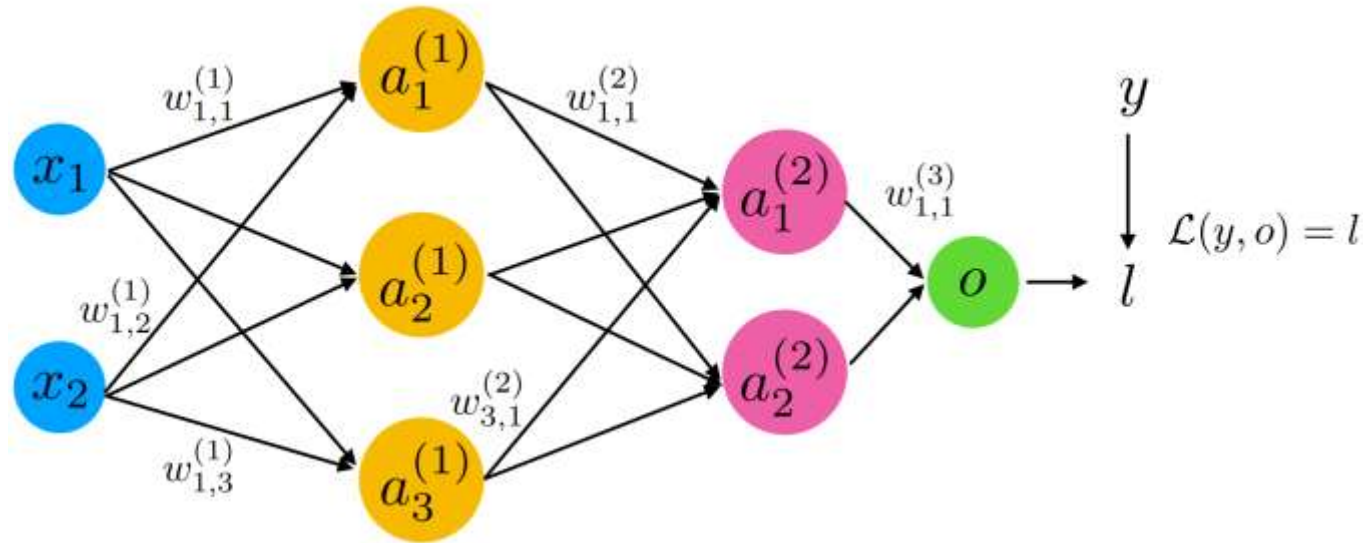
Gradient

Reverse Accumulation

Assume $L = (\langle \mathbf{x}, \mathbf{w} \rangle - y)^2$



Gradient Vanishing Problem



Suppose sigmoid activation function is applied:

$$\frac{\partial l}{\partial w_{1,1}^{(1)}} = \frac{\partial l}{\partial o} \cdot \frac{\partial o}{\partial a_1^{(2)}} \cdot \frac{\partial a_1^{(2)}}{\partial a_1^{(1)}} \cdot \frac{\partial a_1^{(1)}}{\partial w_{1,1}^{(1)}} + \frac{\partial l}{\partial o} \cdot \frac{\partial o}{\partial a_2^{(2)}} \cdot \frac{\partial a_2^{(2)}}{\partial a_1^{(1)}} \cdot \frac{\partial a_1^{(1)}}{\partial w_{1,1}^{(1)}}$$

$$a_1^{(2)} = \sigma(z_1^{(2)}) = \frac{1}{1 + \exp(-z_1^{(2)})}$$

$$z_1^{(2)} = w_{1,1}^{(2)} a_1^{(1)} + w_{2,1}^{(2)} a_2^{(1)} + w_{3,1}^{(2)} a_3^{(1)}$$

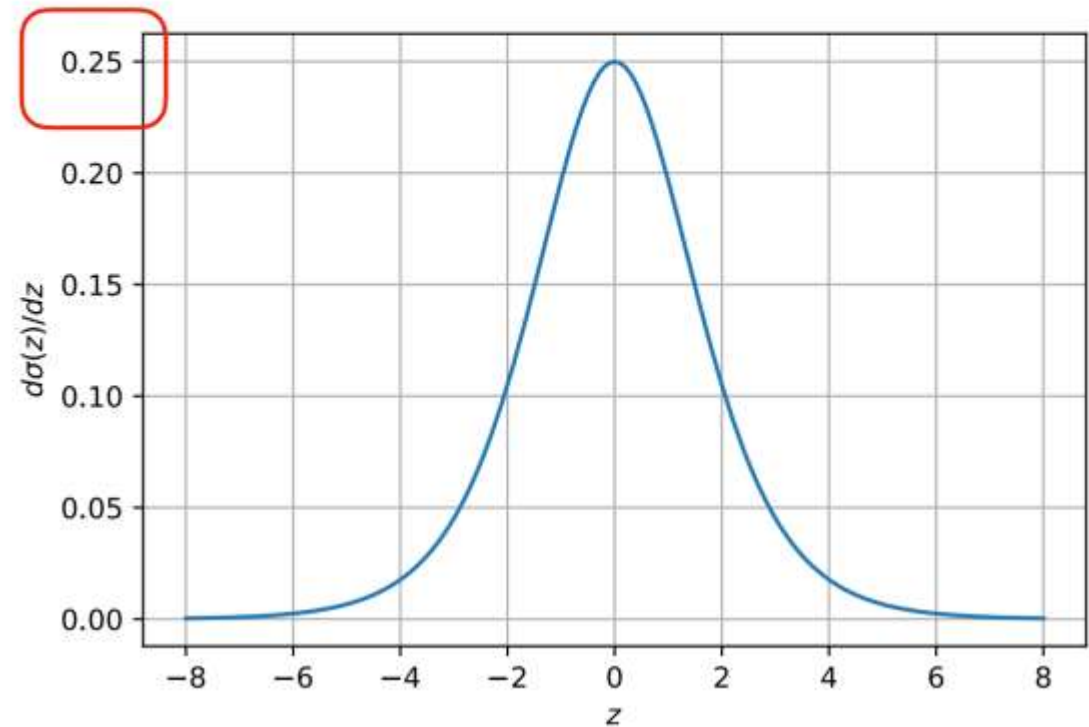
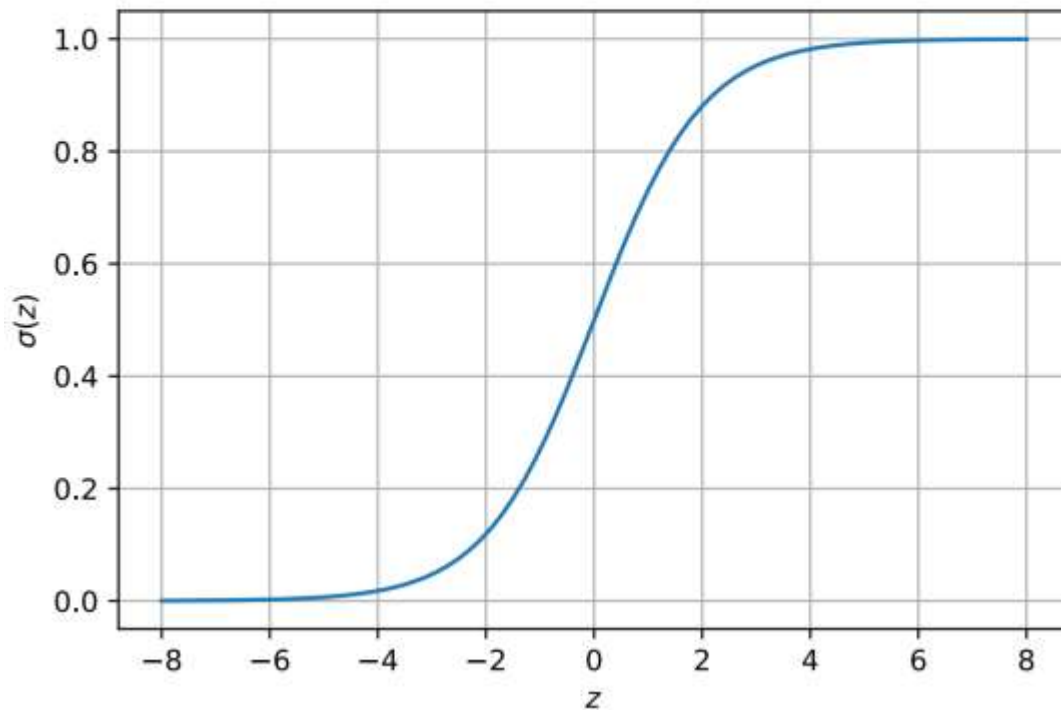
Gradient Vanishing Problem

The highest gradient of sigmoid is **0.25**

Suppose 10 layers using sigmoid is applied: $0.25^{10} \approx 10^{-6}$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\frac{d}{dz}\sigma(z) = \frac{e^{-z}}{(1 + e^{-z})^2} = \sigma(z)(1 - \sigma(z))$$



Random Initialization with the Same Variance

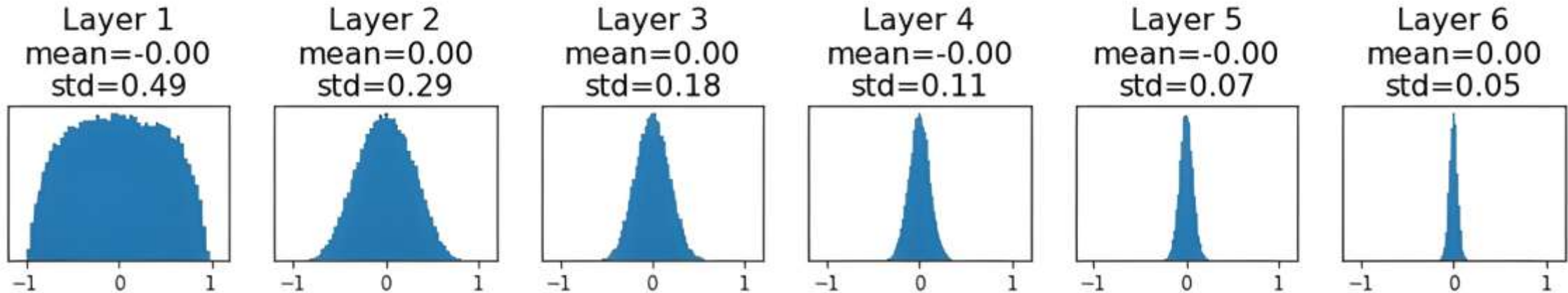
```
dims = [4096] * 7      Forward pass for a 6-layer
hs = []                net with hidden size 4096
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = 0.01 * np.random.randn(Din, Dout)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

All activations tend to zero
for deep network layers

How about gradients?

$$x_{n+1} = \tanh(z_{n+1}) \quad z_{n+1} = w_n x_n + b$$

$$\frac{dx_{n+1}}{dw_n} = \frac{dx_{n+1}}{dz_{n+1}} \cdot x_n \approx 0$$



Optimizers

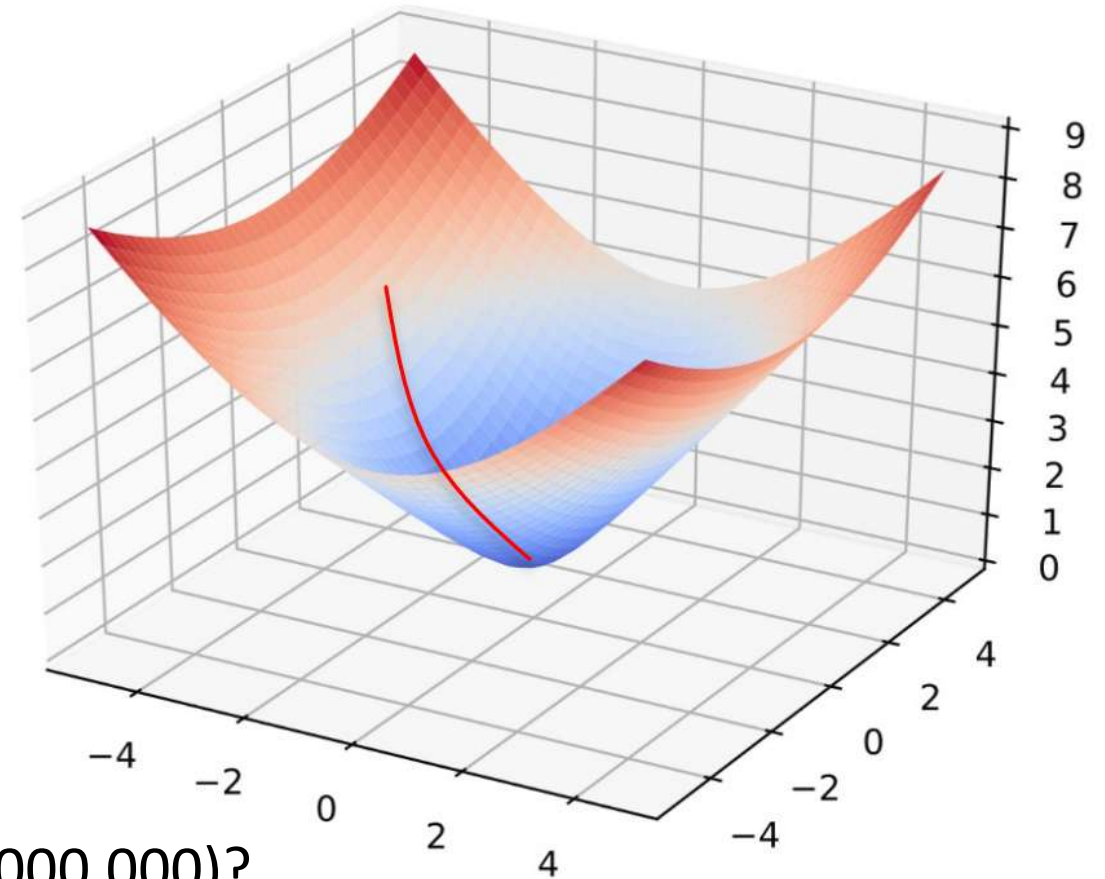
Gradient Descent

Loss function

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N l(x_i, y_i, \theta)$$

For $t = 1, \dots, T$:

$$\theta^{(t)} \leftarrow \theta^{(t-1)} - \alpha \nabla_{\theta} J(\theta)$$



What if we have 1 million training samples ($N > 1,000,000$)?
Gradient Descent takes too much computation.

Stochastic Gradient Descent

For $t = 1, \dots, T$:

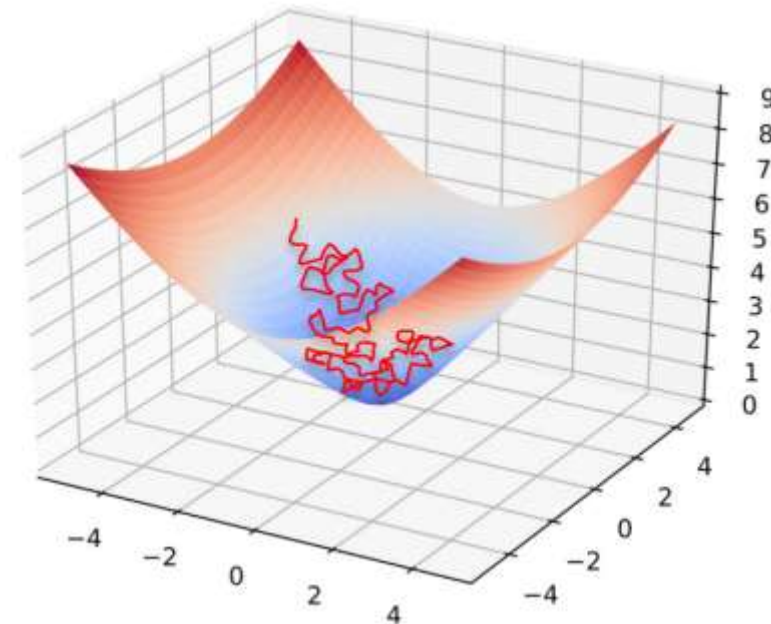
Sample K data points as a batch D

Computes Loss on the batch $J(\theta) = \frac{1}{K} \sum_{x_i \in D} l(x_i, y_i, \theta)$

Update parameters $\theta^{(t)} \leftarrow \theta^{(t-1)} - \alpha \nabla_{\theta} J(\theta)$

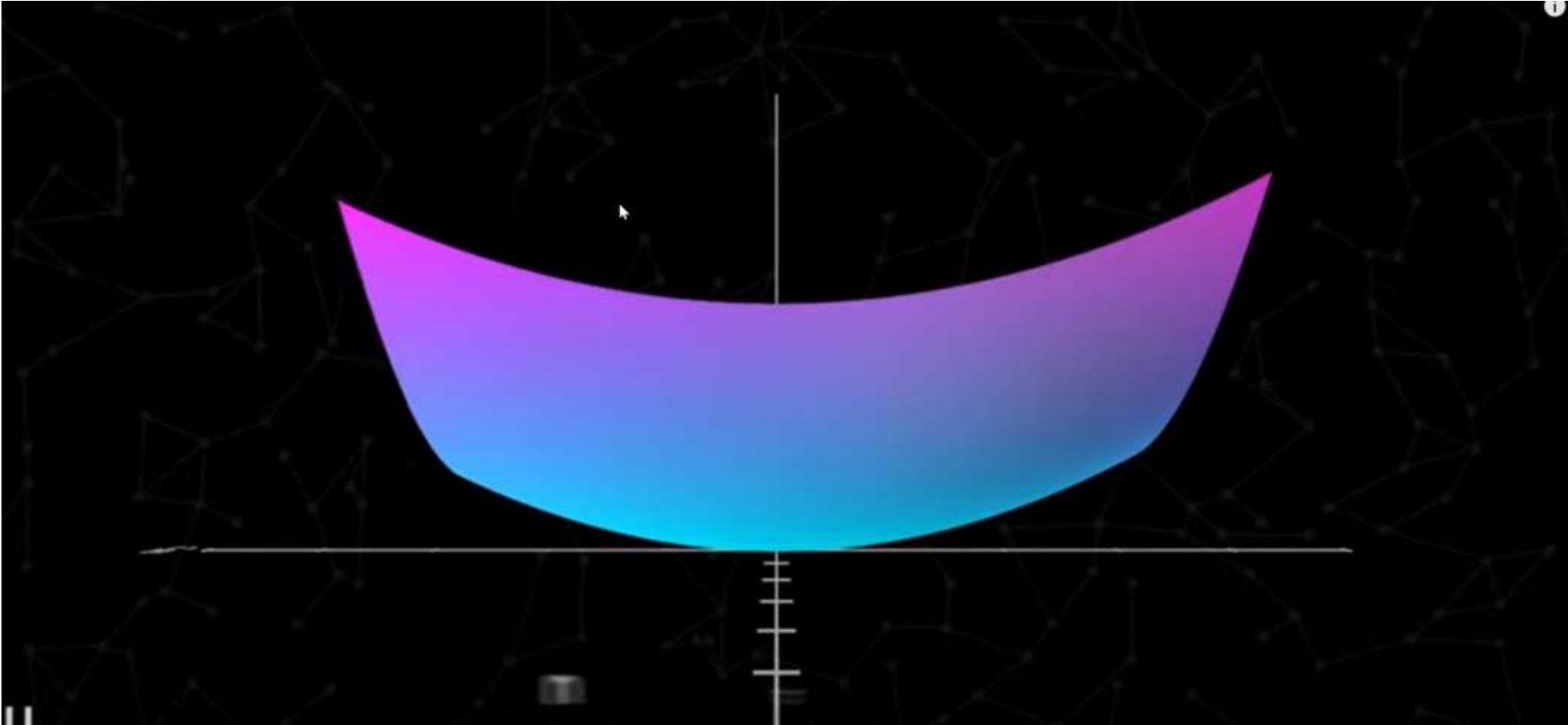
The batch size could be much smaller than the training data size

- 32, 64, 128, 256, 512, 1024,...
- Larger batch size is better



Optimization: Problem #1 with SGD

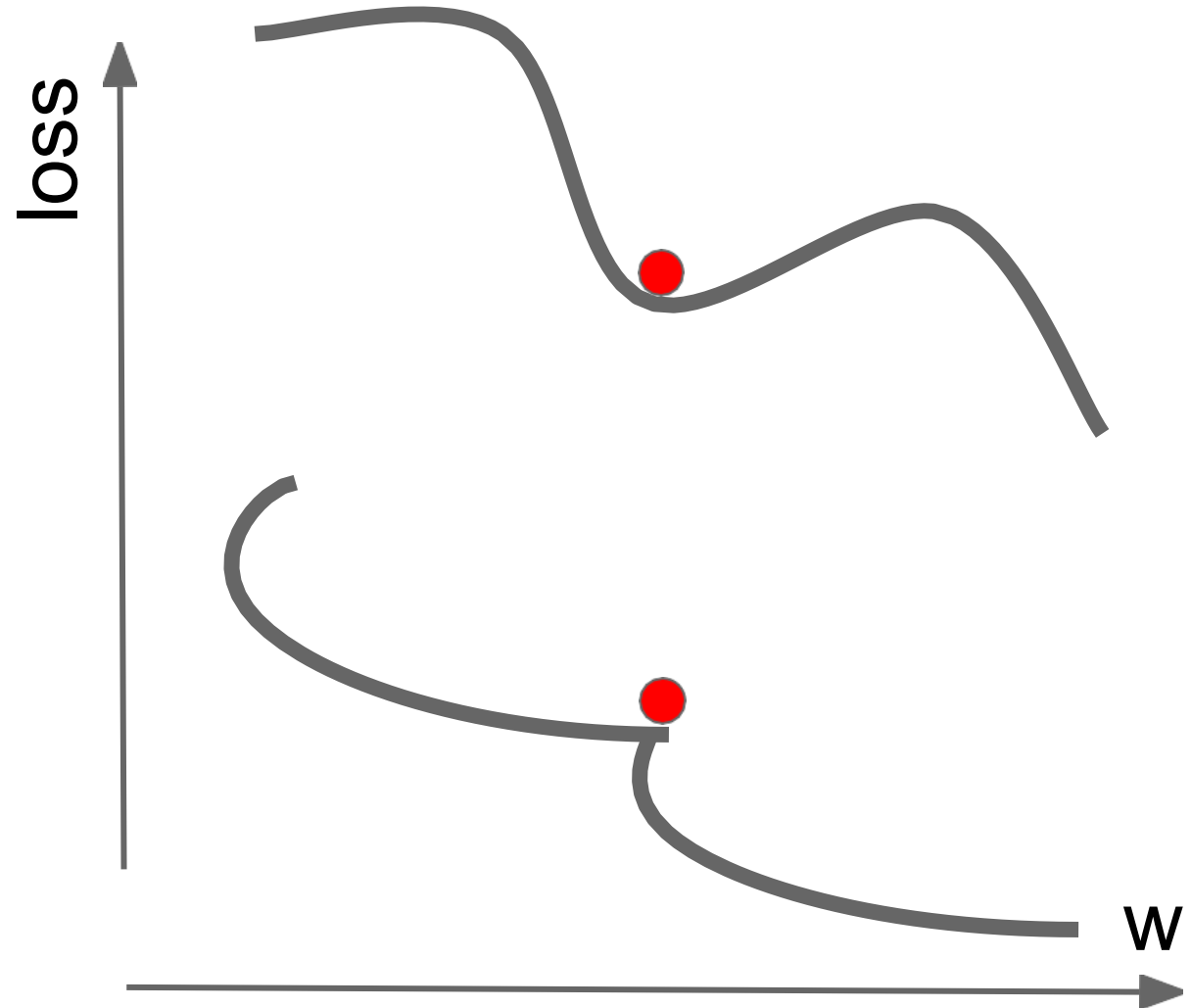
Requires a lot of iterations when gradients are small



Optimization: Problem #2 with SGD

What if the loss function has a **local minima** or **saddle point**?

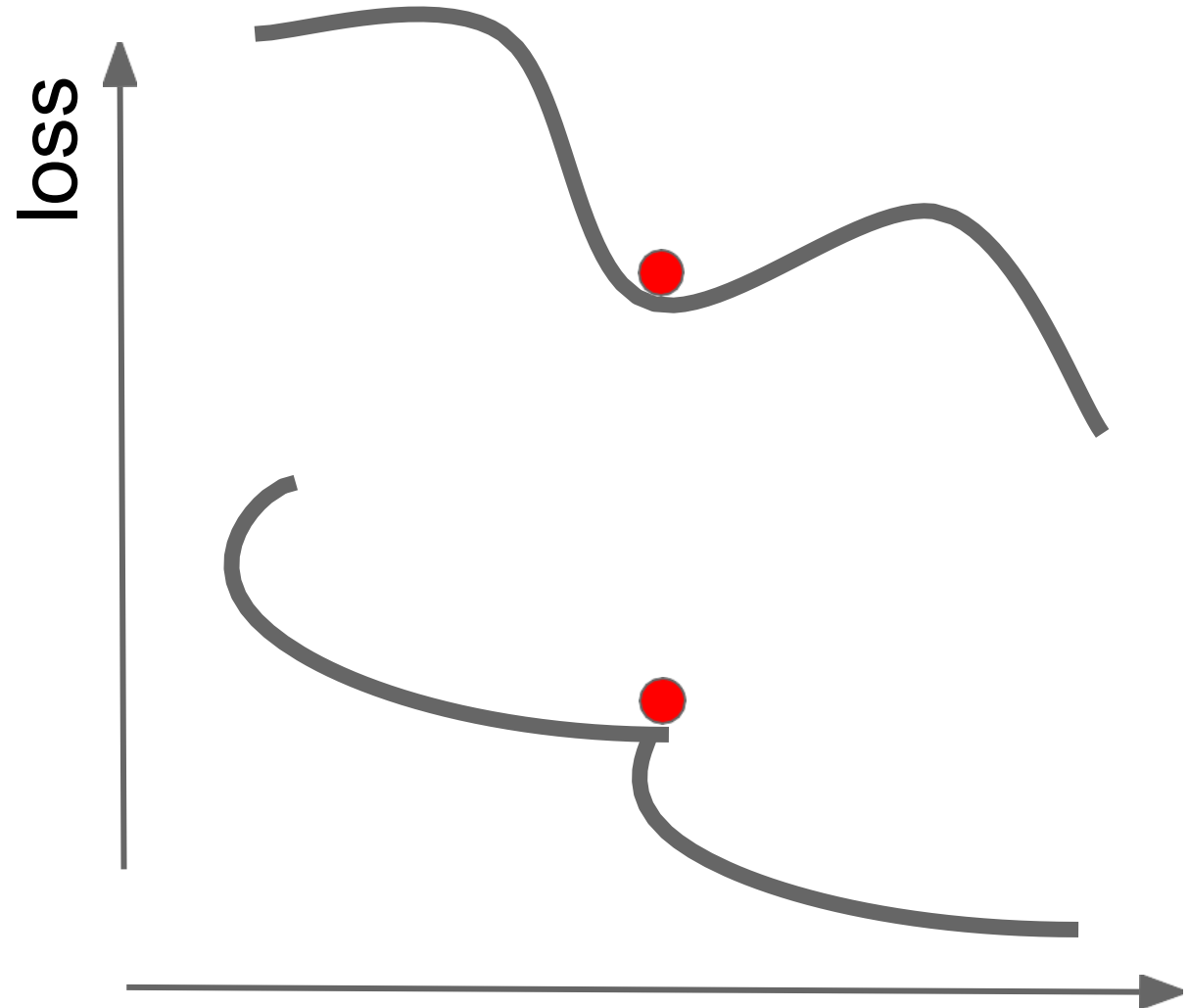
Zero gradient,
gradient descent
gets stuck



Optimization: Problem #2 with SGD

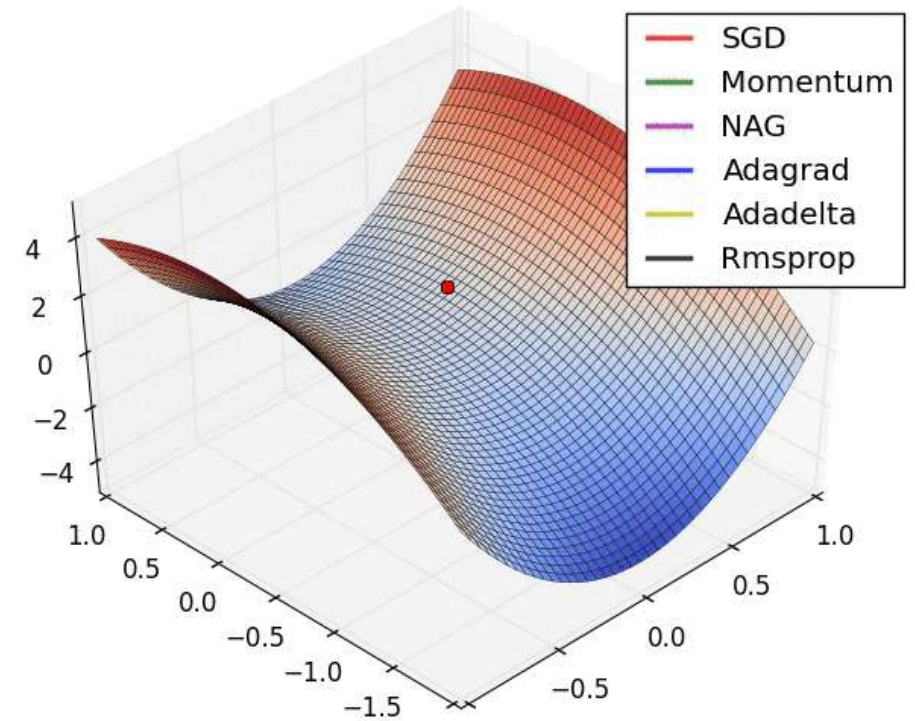
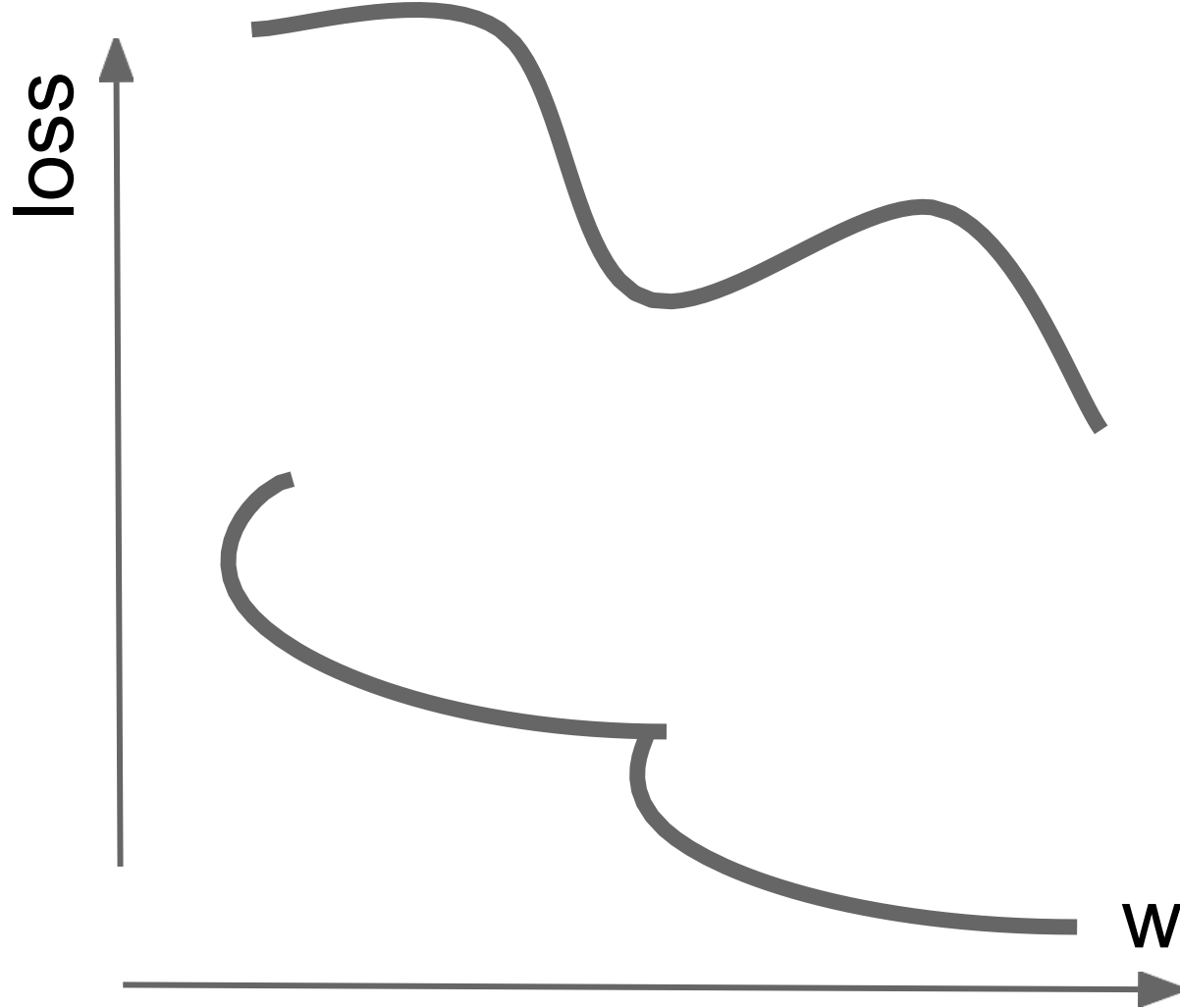
What if the loss function has a **local minima** or **saddle point**?

Saddle points much more common in high dimension



Optimization: Problem #2 with SGD

Stuck in local minimum or saddle point



What are the Solutions?

1. Momentum (动量法)

1.1 Core Idea

Momentum accelerates gradient descent by **accumulating past gradients** to dampen oscillations in steep or noisy regions. It introduces a "velocity" term that smooths the optimization path.

1.2 Mathematical Formulation

• Update Rule:

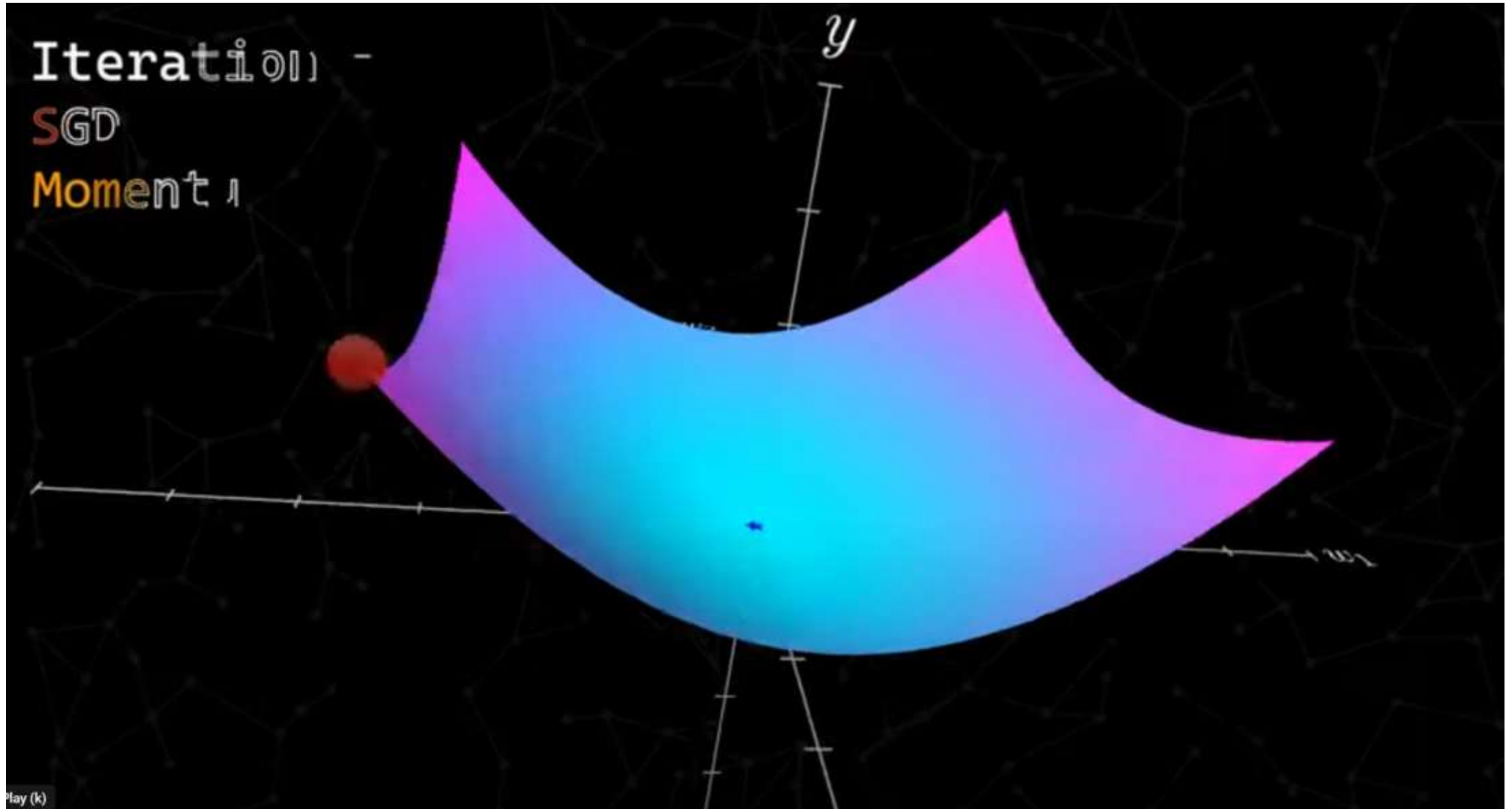
Momentum

$$\begin{cases} v_t = \beta v_{t-1} + (1 - \beta) \nabla_{\theta} J(\theta_t) \\ \theta_{t+1} = \theta_t - \alpha v_t \end{cases}$$

- v_t : Velocity vector (weighted average of past gradients).
- β : Momentum coefficient (typically **0.9**).
- α : Learning rate.

Advantages	Disadvantages
Faster convergence in shallow regions.	May overshoot minima if β is too high.
Reduces oscillations in steep valleys.	Requires tuning β and α .

With momentum, it converges much faster.



2. Nesterov Accelerated Gradient (NAG, 涅斯捷罗夫加速梯度)

2.1 Core Idea

NAG improves Momentum by "looking ahead" before computing the gradient. Instead of evaluating the gradient at the current position (θ_t), it uses a **future position** ($\theta_t + \beta v_{t-1}$) to adjust the update direction.

2.2 Mathematical Formulation

- Update Rule:

$$\begin{cases} v_t = \beta v_{t-1} + (1 - \beta) \nabla_{\theta} J(\theta_t + \beta v_{t-1}) \\ \theta_{t+1} = \theta_t - \alpha v_t \end{cases}$$

Gradients of “looking ahead”

- The gradient is computed at $\theta_t + \beta v_{t-1}$ (future position).

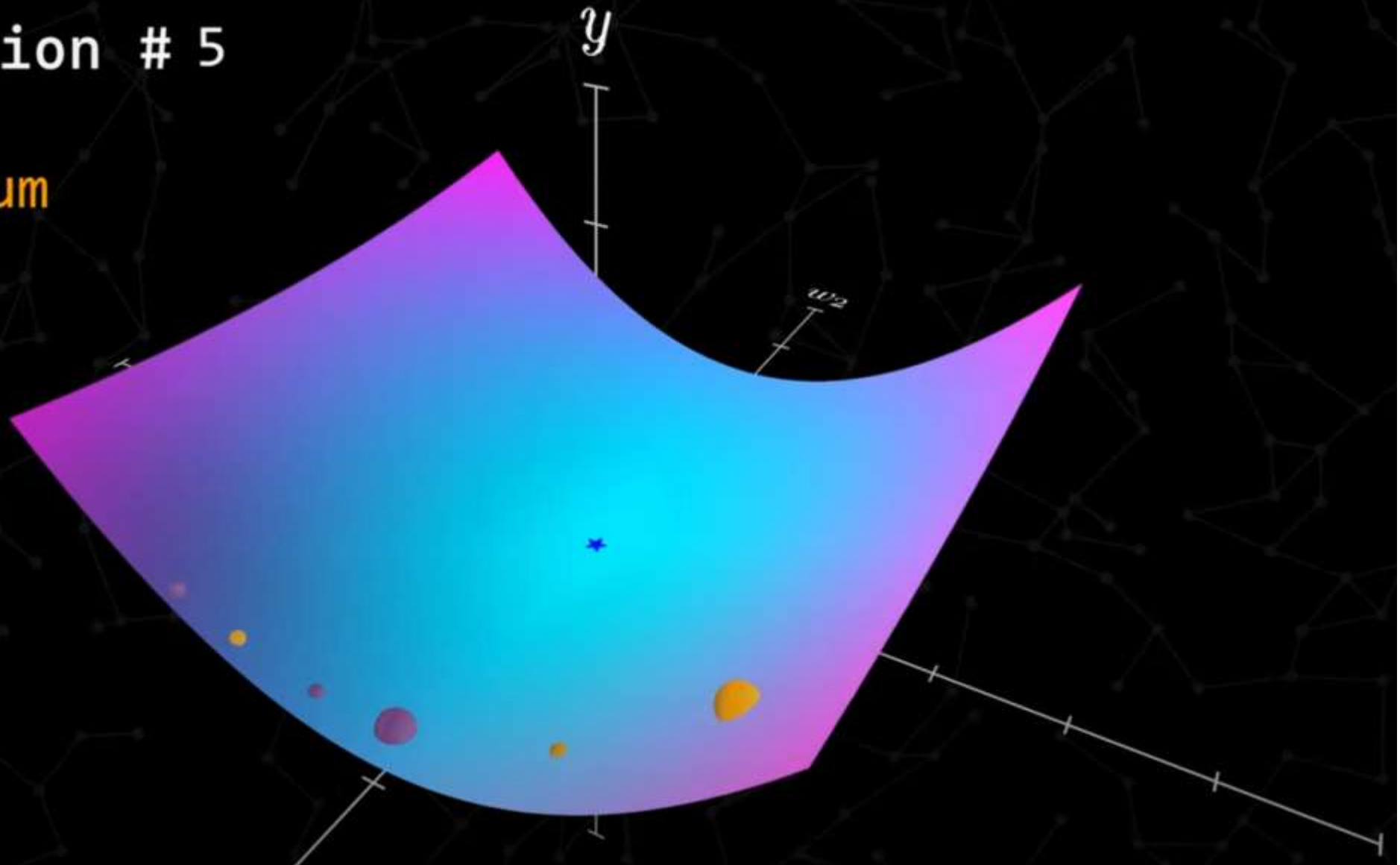
Advantages	Disadvantages
More accurate gradient estimation.	Slightly more computation per step.
Reduces oscillations better than Momentum.	Still sensitive to hyperparameters.

Iteration # 5

NAG

Momentum

Play (k)



Adagrad

Adagrad adapts learning rates per-parameter based on historical gradient magnitudes. It assigns smaller updates to frequent features and larger updates to sparse features.

Mathematical Formulation

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{G_t + \epsilon}} \odot \nabla_{\theta} J(\theta_t)$$

“personalized” learning rate

- G_t : Diagonal matrix of sum of squared past gradients ($G_t = G_{t-1} + (\nabla_{\theta} J(\theta_t))^2$).
- ϵ : Small constant (e.g., $1e-8$) to avoid division by zero.

Key Properties

Advantages	Disadvantages
Works well for sparse data (e.g., NLP).	Learning rates may vanish over time.
No manual learning rate tuning needed.	Performs poorly for non-convex problems.

RMSProp

RMSProp fixes Adagrad's aggressive learning rate decay by replacing the sum of squared gradients with an **exponentially weighted moving average (EWMA)**.

Mathematical Formulation

$$\begin{cases} E[g^2]_t = \beta E[g^2]_{t-1} + (1 - \beta)(\nabla_{\theta} J(\theta_t))^2 \\ \theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{E[g^2]_t + \epsilon}} \odot \nabla_{\theta} J(\theta_t) \end{cases}$$

- $E[g^2]_t$: EWMA of squared gradients (β typically 0.9).

Add an exponentially weighted moving average

Key Properties

Advantages	Disadvantages
Solves Adagrad's vanishing LR issue.	Still requires tuning β and α .
Works well for non-stationary objectives (e.g., RNNs).	May need Momentum for some tasks.

Adam

It combines **Momentum (1st moment)** and **RMSProp (2nd moment)** for adaptive learning rates and smooth convergence.

Mathematical Formulation

$$\begin{cases} m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla_{\theta} J(\theta_t) & \text{(1st moment)} \\ v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla_{\theta} J(\theta_t))^2 & \text{(2nd moment)} \\ \hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} & \text{(bias correction)} \\ \theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon} \odot \hat{m}_t \end{cases}$$

- β_1 : Momentum decay (default 0.9).
- β_2 : Squared gradient decay (default 0.999).

Key Properties

Advantages	Disadvantages
Fast convergence and stability.	More hyperparameters to tune (β_1, β_2, α).
Works well for deep learning (CNNs/Transformers).	May underperform if defaults are misused.

Optimizer	Mechanism	When to Use	Strengths	Weaknesses
Momentum	Gradient smoothing with velocity.	General convex optimization.	Reduces oscillations in valleys.	May overshoot minima.
NAG	“Looks ahead” before gradient step.	High-curvature landscapes.	Faster convergence than Momentum.	Slightly more computation per step.
Adagrad	Per-parameter adaptive LR (sum of squares).	Sparse data (e.g., NLP).	No manual LR tuning for sparse features.	LR vanishes over time.
RMSProp	EWMA of squared gradients.	Non-stationary tasks (e.g., RNNs).	Fixes Adagrad’s LR decay.	Needs tuning β .
Adam	Momentum + RMSProp + bias correction.	Deep learning (default choice).	Fast and stable convergence.	Hyperparameter-sensitive.



TED

Let's combine all together

Multi-Layer Perception

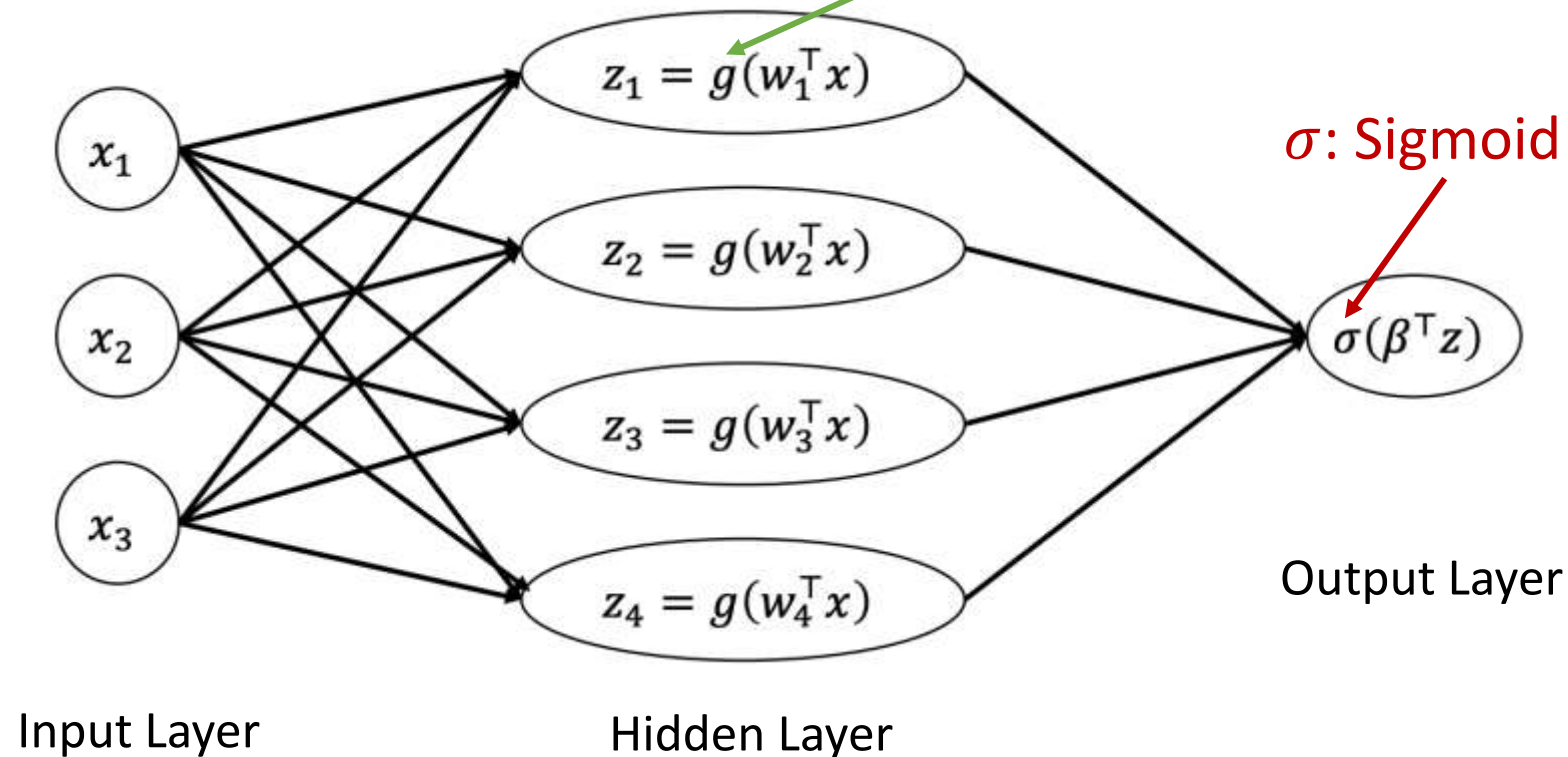
MLP is composed of multiple layers of neural networks

For binary classification: $f(\mathbf{x}) = \sigma(\beta^T g(\mathbf{W}\mathbf{x}))$

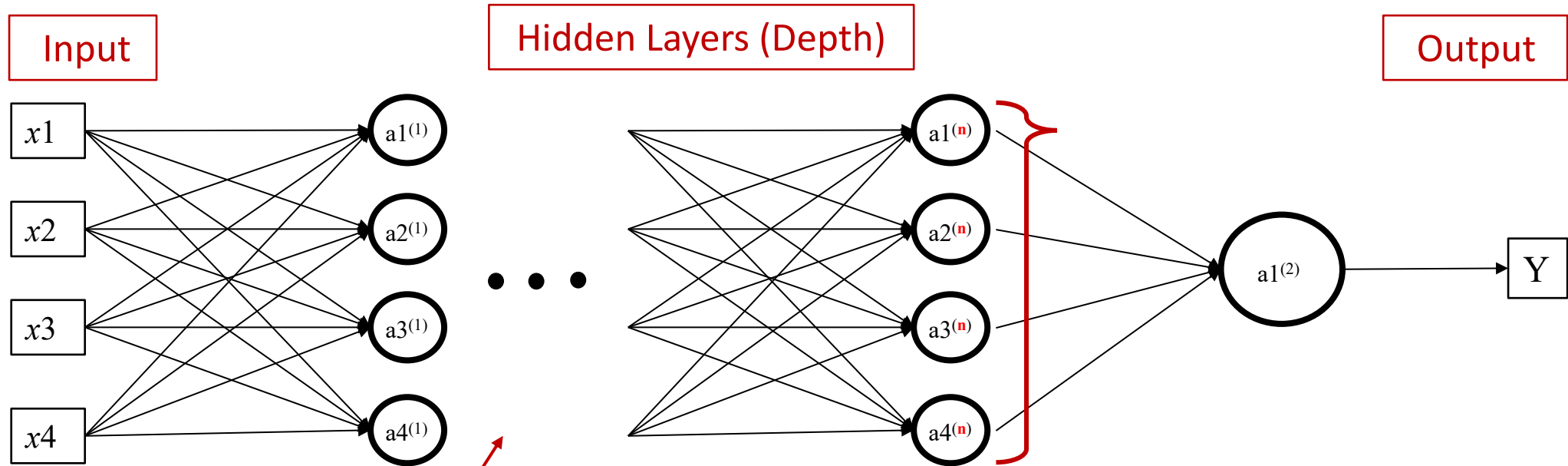
g : Non-linear activation function

σ : Sigmoid activation

MLP can stack many hidden layers

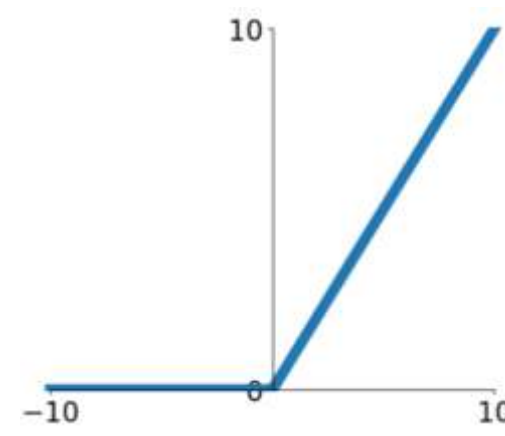


Architecture



Hidden Layer Activation Function

ReLU: $f(x) = \max(0, x)$



Optimization

Loss function to minimize

Regression: MSE Loss $\frac{1}{N} \sum_{i=1}^N (f(\mathbf{x}_i) - y_i)^2$

Classification: Cross Entropy Loss $\frac{1}{N} \sum_{i=1}^K H(f(\mathbf{x}_i), y_i)$

$$H(f(\mathbf{x}), y) = -(y \log f(\mathbf{x}) + (1 - y) \log(1 - f(\mathbf{x})))$$

Regularization (avoid overfitting)

L1 Norm: $\min_{\mathbf{w}} L(\mathbf{w}) + \lambda \sum_i |\mathbf{w}|$

Induce sparse weights

L2 Norm: $\min_{\mathbf{w}} L(\mathbf{w}) + \lambda \sum_i |\mathbf{w}|^2$

Mostly applied
Also known as weight decay



Optimization

Neural Network Weight initialization

- **Xavier initialization:** Maintain output variance after multiple layers
- **Kaiming initialization:** Designed for ReLU, current default choice

Optimization algorithm: Gradient Descent

Gradient Descent Algorithm

1. Initialize $\mathbf{w}^{(0)}$ by random values

2. For $t = 1, \dots, T$:

$$\mathbf{w}^{(t)} \leftarrow \mathbf{w}^{(t-1)} - \alpha \nabla_{\mathbf{w}} L(\mathbf{w})$$

Optimizer: ADAM



Hyperparameter Tuning and Evaluation

Typical hyperparameters to tune: coefficient of regularization, learning rate, number of layers (depth), hidden dimension (width), batch size, ...

Split dataset:



Evaluation metrics:

$$\text{Accuracy} = \frac{TP + TN}{N} \quad \text{Recall} = \frac{TP}{TP + FN} \quad \text{Precision} = \frac{TP}{TP + FP} \quad \text{F1 score, ROC-AUC}$$

1. Train models with different hyperparameters on training data
2. Select hyperparameter that performs best on validation data
3. Report the model performance on test data

Challenges of Training MLP

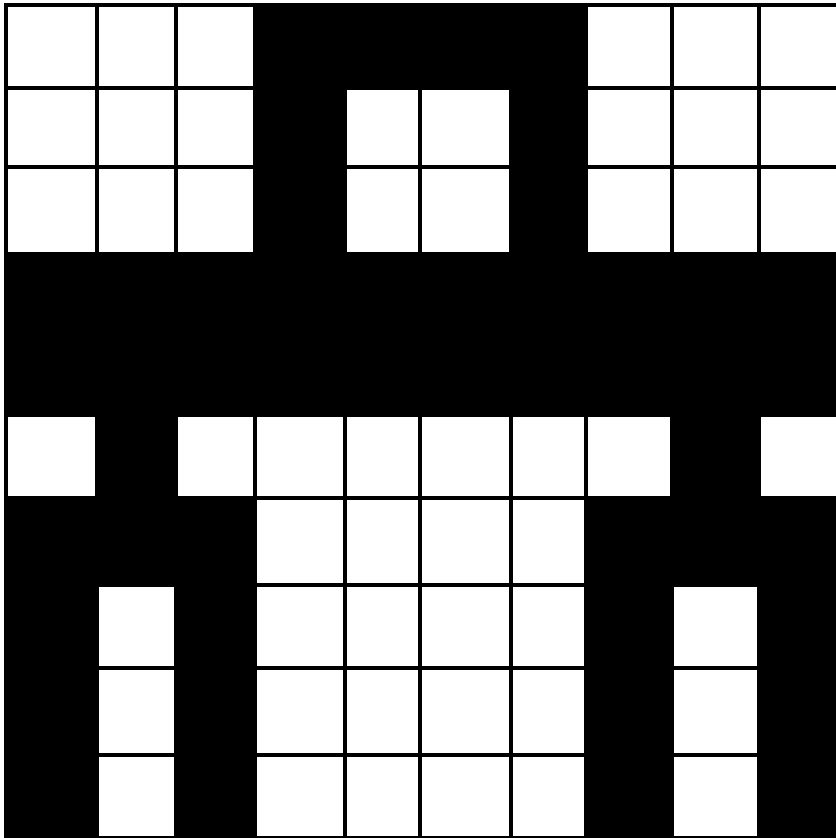
- No closed form solution, non-convex optimization
- Requires gradient decent which requires significant computation
- Gradient vanishing/exploding happens in deep neural networks
 - Choose a good initialization
 - Set a right optimizer and hyperparameters especially learning rate
 - Use the ReLU as activation function
 - Other designs of neural networks such as ResNet. Will be discussed later

Introduction to Convolutional Neural Networks



How is image represented in computer?

Black and white image



Represented
by matrix

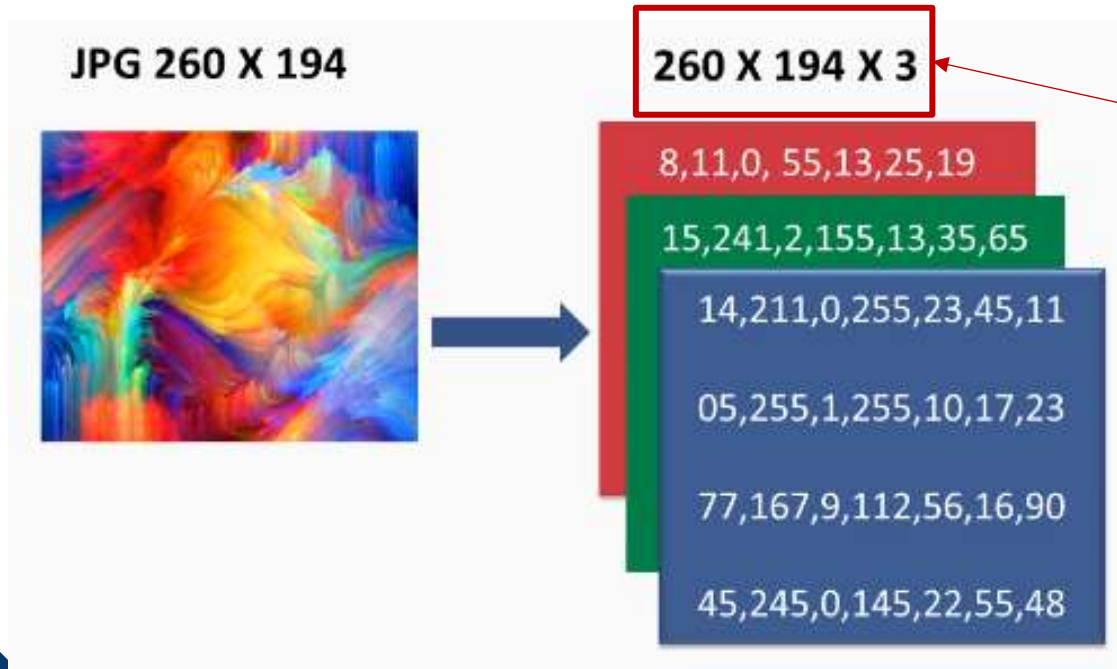
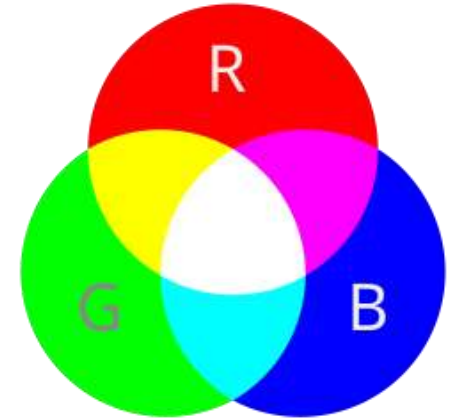


Pixel

0	0	0	1	1	1	1	0	0	0
0	0	0	1	0	0	1	0	0	0
0	0	0	1	0	0	1	0	0	0
1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1
0	1	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	1	1	1
1	0	1	0	0	0	0	1	0	1
1	0	1	0	0	0	0	1	0	1
1	0	1	0	0	0	0	1	0	1

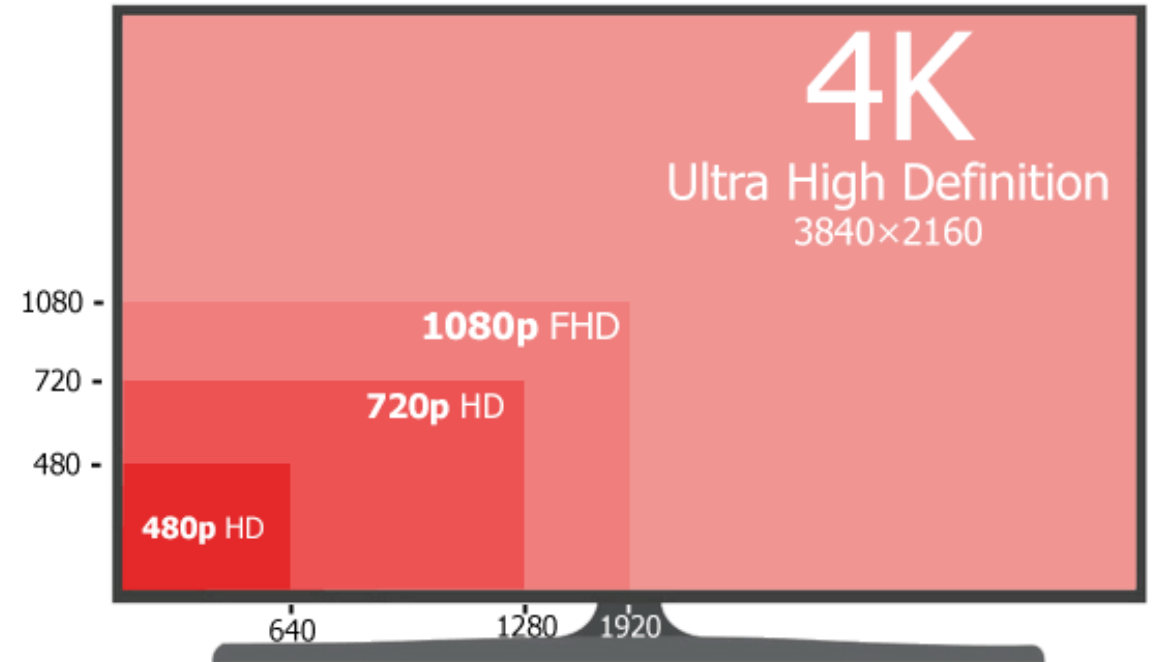
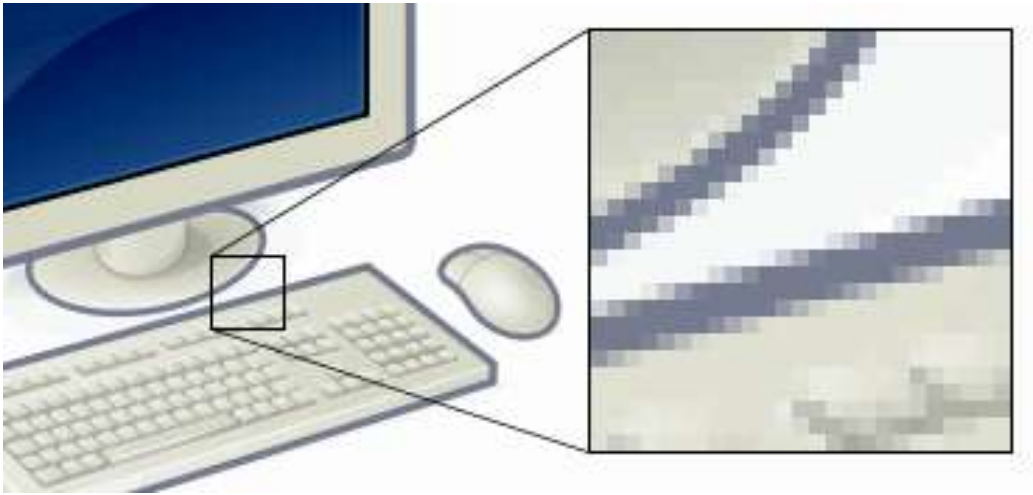
How is image represented in computer?

Red, **Green** and **Blue** can compose to any color



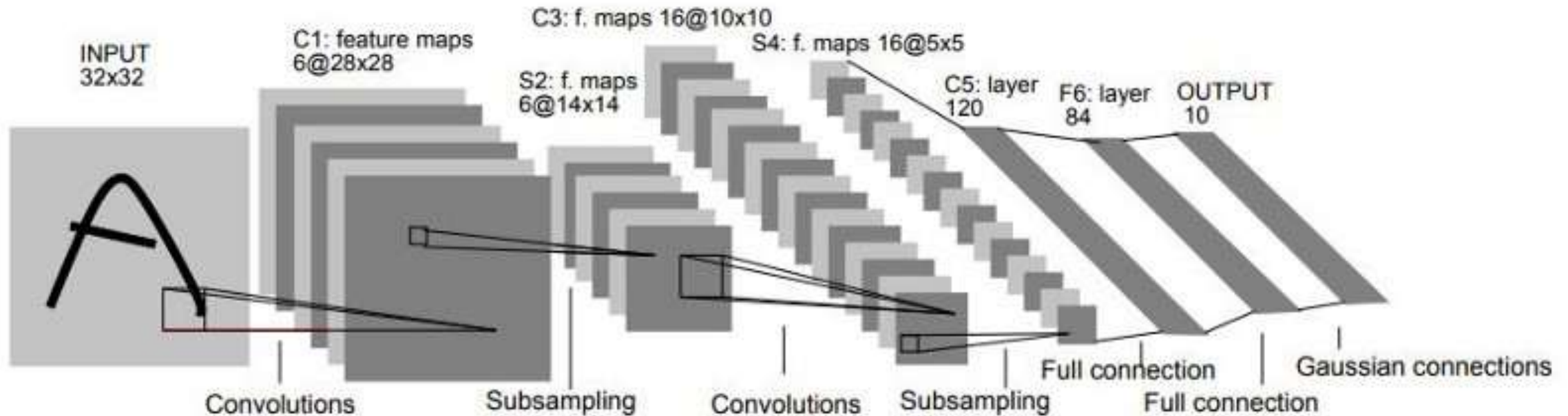
Width: 260 pixels
Height: 194 pixels
Channel: 3 (RGB)

How is image represented in computer?



History of CNN

Gradient-based learning applied to document recognition
[LeCun, Bottou, Bengio, Haffner 1998]

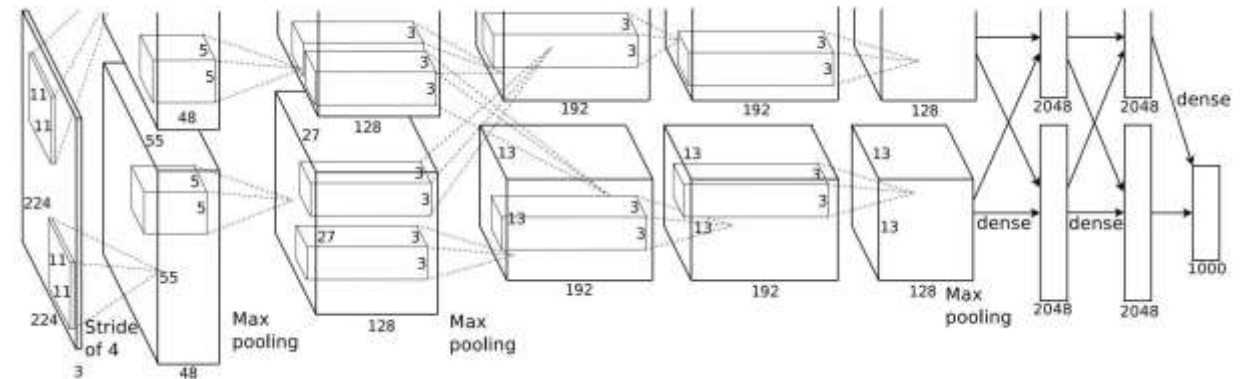


Start of Deep Learning

Feifei Li created Imagenet: largest image classification dataset at that time



ImageNet Classification with Deep Convolutional Neural Networks
[Krizhevsky, Sutskever, Hinton, 2012]

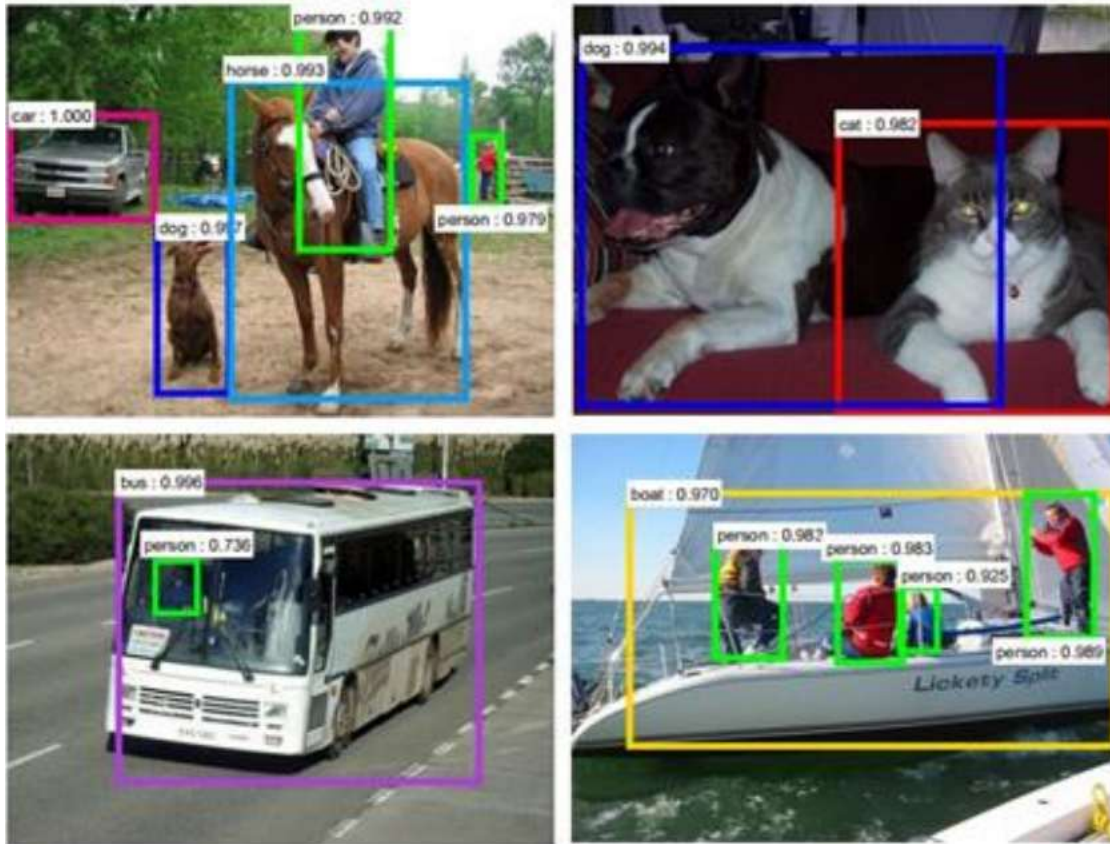


AlexNet



2012 – 2020: ConvNets dominate all vision tasks

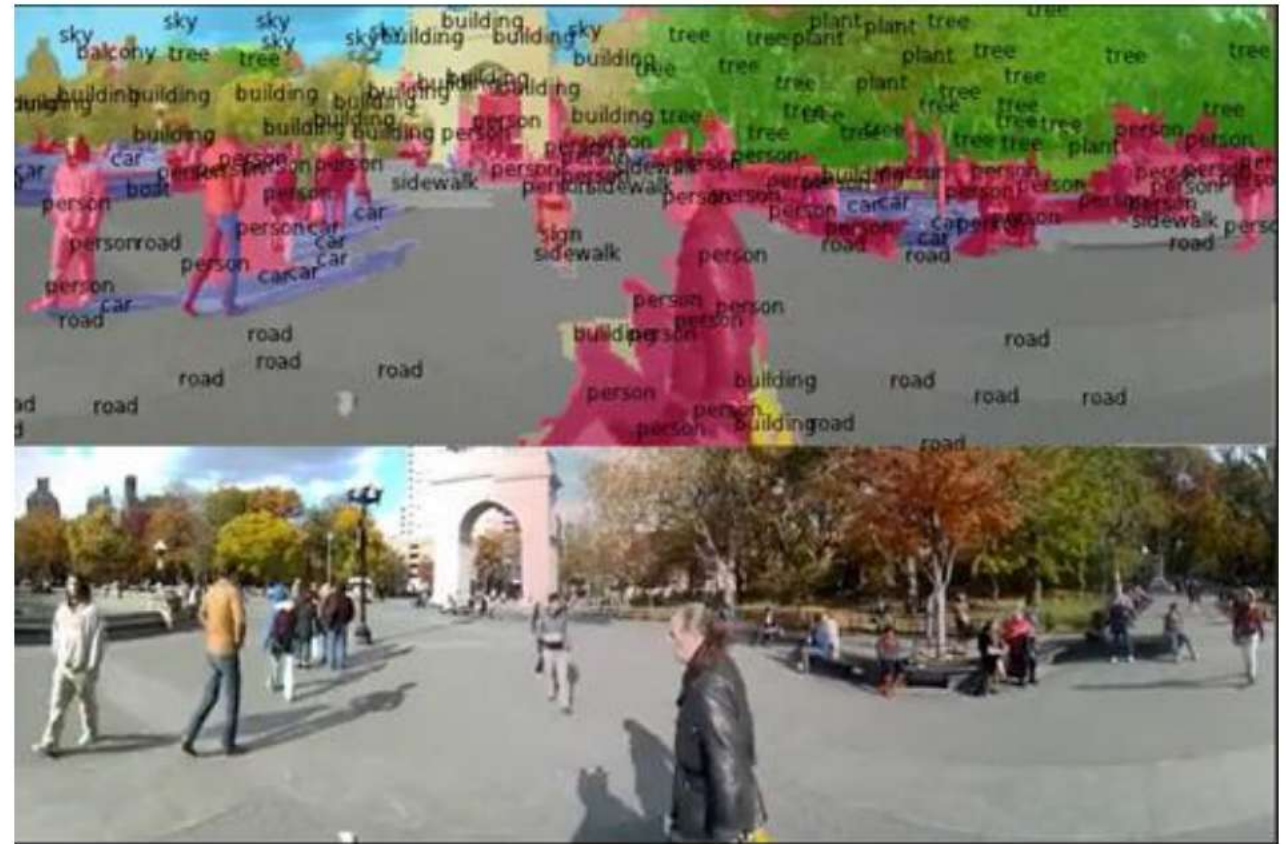
Detection



Figures copyright Shaoqing Ren, Kaiming He, Ross Girshick, Jian Sun, 2015. Reproduced with permission.

Faster R-CNN: Ren, He, Girshick, Sun 2015]

Detection



Figures copyright Clement Farabet, 2012. Reproduced with permission.

[Farabet et al., 2012]

2012 – 2020: ConvNets dominate all vision tasks

Image Captioning



A white teddy bear sitting in the grass



A man in a baseball uniform throwing a ball



A woman is holding a cat in her hand



A man riding a wave on top of a surfboard



A cat sitting on a suitcase on the floor



A woman standing on a beach holding a surfboard

All images are CC0 Public domain:
<https://pixabay.com/en/luggage-antique-cat-1643010/>
<https://pixabay.com/en/teddy-plush-bears-cute-teddy-bear-1623436/>
<https://pixabay.com/en/surf-wave-summer-sport-litoral-1668716/>
<https://pixabay.com/en/woman-female-model-portrait-adult-983967/>
<https://pixabay.com/en/handstand-lake-meditation-496008/>
<https://pixabay.com/en/baseball-player-shortstop-infield-1045263/>

Captions generated by Justin Johnson using [NeuralTalk2](#)

[Vinyals et al., 2015]
[Karpathy and Fei-Fei, 2015]

2012 – 2020: ConvNets dominate all vision tasks

Text-to-Image Generation

Rombach et al, "High-Resolution Image Synthesis with Latent Diffusion Models", CVPR 2022



A zombie in the style of Picasso

An image of a half mouse half octopus

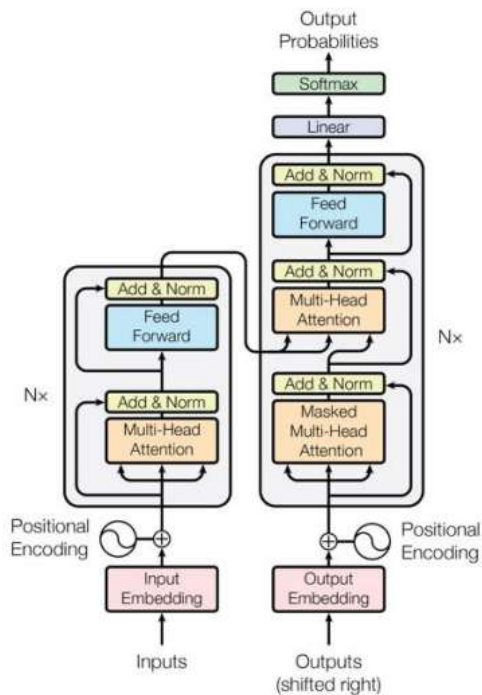
A painting of a squirrel eating a burger

A watercolor painting of a chair that looks like an octopus

A shirt with the inscription: "I love generative models!"

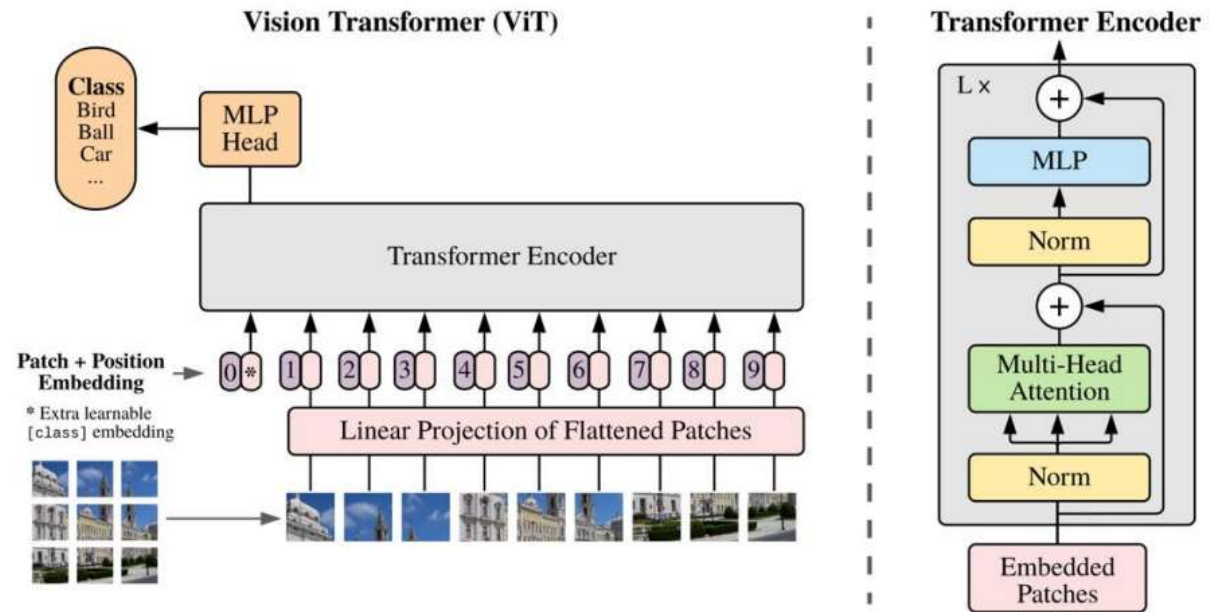
2021 - Present: Transformers have taken over

2017: Transformers
for language tasks



Vaswani et al, "Attention is all you need", NeurIPS 2017

2021: Transformers
for vision tasks



Dosovitskiy et al, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale", ICLR 2021